

## MODULE 3 · STUDY GUIDE

# The Agentic Leap

Architecture · Reasoning · Protocols · Orchestration

An agent is RAG plus tool use plus control flow. This guide takes you from the bare agent loop to a diagrammable LangGraph state machine — the same practitioner depth as the Module 1 and 2 guides, built around the Filing Analyst Copilot.

Agent loop

ReAct

Chain-of-Thought

MCP & A2A

LangChain

LangGraph

## CONTENTS

# What's inside

Four concepts, read in order — each builds on the last. The straight line of Module 2's RAG pipeline bends into a loop, the loop gets a brain, the brain gets standard plugs, and the whole thing becomes a graph you can draw.

|                                                                                                          |                                                    |    |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------|----|
| 1                                                                                                        | Agentic Architecture — the loop and its four parts | 03 |
| A model in a loop with tools · model / tools / memory / control flow · why control flow is the hard part |                                                    |    |
| 2                                                                                                        | Reasoning & Planning — making the next move well   | 06 |
| Chain-of-Thought · ReAct · plan-and-execute vs. plan-as-you-go · reflection                              |                                                    |    |
| 3                                                                                                        | Protocols — MCP & A2A                              | 09 |
| The N×M problem · MCP host/client/server · A2A Agent Cards & Tasks · the one-line distinction            |                                                    |    |
| 4                                                                                                        | Orchestration — LangChain & LangGraph              | 12 |
| Building blocks vs. the black box · state / nodes / edges · the loop you can draw                        |                                                    |    |

### HOW TO USE THIS GUIDE

Read it once front-to-back for the arc, then keep it next to the build. The **Mental Model** boxes are the things to be able to say out loud in an interview; the code blocks are the things to be able to write at a whiteboard.

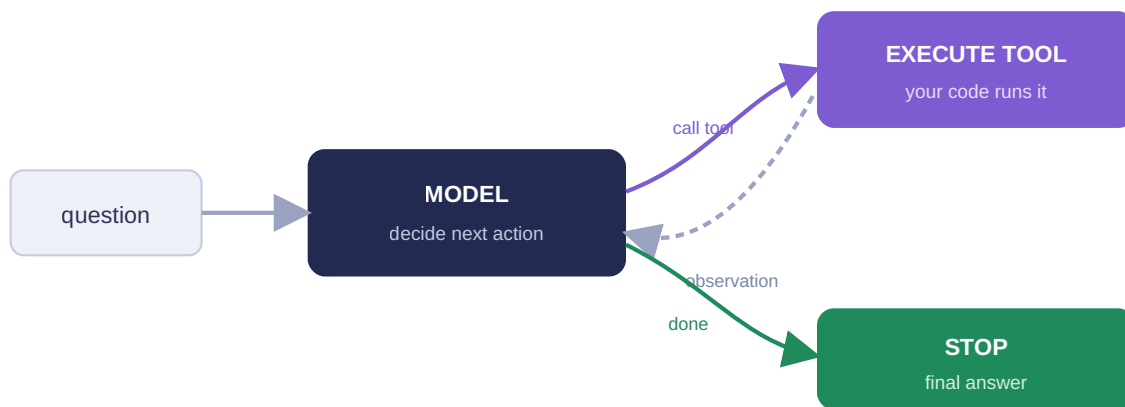
## CONCEPT 1

# Agentic Architecture

**The one idea:** an agent is a loop. A model decides on an action, something executes it, the result goes back to the model, and it decides again — repeating until the model decides it's done.

## From a straight line to a loop

In Module 2 you built a pipeline: question → retrieve → stuff context → generate → answer. It runs once and it's over; the model never gets a second look. An agent breaks that straight line into a loop where the model steers. Each turn its job is small — *given everything I know so far, what is the single next thing to do?* It either picks a tool to call, or decides it has enough to answer.



The agent loop: reason → act → observe, until the model chooses to stop.

### MENTAL MODEL

RAG retrieves *once* and the developer fixed the order. An agent retrieves (and computes, and looks up) *as many times as it needs*, and the model decides the order at runtime. The cost of that flexibility is unpredictability — which is why most of this module is really about *controlling* the loop.

## The four moving parts

Every agent, however it's dressed up, is built from four things:

- ▶ **The model (the brain)** — given the conversation plus the available tools, it outputs either a tool call or a final answer. The only "intelligent" part; it never runs anything itself.
- ▶ **Tools (the hands)** — functions the model can ask to run, each with a name, description, and typed argument schema. The model *requests* a call; your code executes it.
- ▶ **Memory (the notebook)** — what carries between turns. Short-term = the running transcript in the context window; long-term = anything persisted across sessions.
- ▶ **Control flow (the rails)** — the logic that decides whether to loop again or stop, handles tool errors, and caps the number of attempts.

## How one turn actually works

Concretely, here's a single pass of the loop using the function-calling interface from Module 1:

```
# Tools: name, description, typed args schema (same as Module 1 function calling)
tools = [{
    "name": "search_filings",
    "description": "Retrieve relevant chunks from SEC filings for a query.",
    "input_schema": {"type": "object",
        "properties": {"query": {"type": "string"}}, "required": ["query"]},
}]

messages = [{"role": "user", "content": "Did revenue grow between the 2022 and 2023 10-K?"}]

while True:
    resp = model.create(messages=messages, tools=tools)
    if resp.stop_reason != "tool_use":
        print(resp.text); break      # model chose to answer — done
    call = resp.tool_call
    result = run_tool(call.name, call.args)  # OUR code executes it
    messages.append({"role": "assistant", "content": resp.raw})
    messages.append({"role": "tool", "content": result})  # feed result back, loop
```

Three things worth burning in:

- ▶ The model **never executes anything** — it emits a request, your `run_tool` runs it. This is the security boundary.
- ▶ The transcript **is** the memory — every turn appends the request and result, which is also why long runs blow the context budget.
- ▶ The loop ends only when the model stops asking for tools. If it never stops, *you* must (the turn cap).

## Why control flow is the hard part

The model is good at "what's the next action." It's bad at the operational stuff: knowing when to give up, not repeating itself, recovering from a tool error instead of crashing, and not running forever. A naive `while True` agent works in demos and falls over in production — no turn cap (loops forever), no error handling (one exception kills the run), no state beyond the transcript, linear only (can't branch). Every one of those is a control-flow problem, not an intelligence problem.

### MENTAL MODEL

The model supplies intelligence; the framework supplies discipline. A good agent system spends most of its code not on the LLM call but on the rails around it — caps, retries, state, and the decision of when to stop.

### PROJECT TIE-IN

The copilot becomes an agent the moment it must answer a question that needs more than one lookup. The Module 2 retrieval pipeline isn't thrown away — it becomes one *tool* ( `search_filings` ) the agent can call. Each module's output becomes a component the next one orchestrates.

## CONCEPT 2

# Reasoning & Planning

**The one idea:** the loop only works if the model's per-turn decision is good. Reasoning techniques make the model *think before it acts*; planning decides how much to figure out up front; reflection lets it catch its own mistakes.

## Chain-of-Thought — think in steps before answering

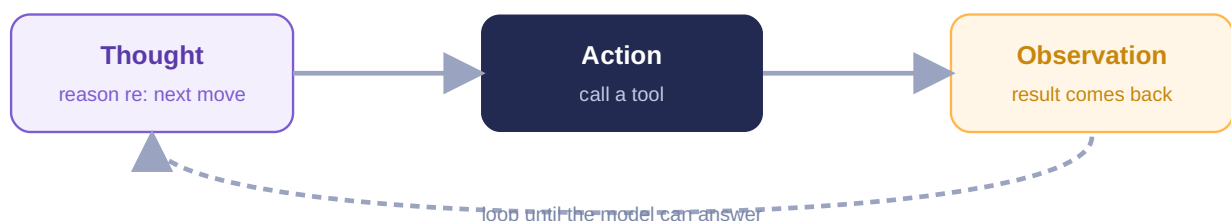
The paired paper for this module (Wei et al. 2022). The finding is almost embarrassingly simple: prompt a model to show its working — "let's think step by step" — and it gets multi-step problems right far more often than if you ask for the answer directly. A transformer does a fixed amount of computation per token, so a hard problem needs more tokens to reach the answer. Writing out the steps gives the model the room — and the compute — to get there. The reasoning text is the model thinking *on paper* because it can't think silently.

### MENTAL MODEL

The model can't pause and ponder. Its only way to "spend more compute" on a hard problem is to generate more tokens. Chain-of-Thought is just permission — and a nudge — to use those tokens for working rather than blurting the answer.

## ReAct — reason, act, observe, repeat

ReAct (Yao et al. 2022) is the pattern nearly every framework implements. It interleaves Chain-of-Thought with tool calls. Each loop turn has three parts: a **Thought** (CoT applied to the next action), an **Action** (a tool call), and an **Observation** (the result, fed back). Interleaving beats a fixed up-front plan because the plan can adapt to what it actually finds.



ReAct = Chain-of-Thought for actions, in a loop. The same loop from Concept 1, with a reasoning step on every turn.

### MENTAL MODEL

ReAct = Chain-of-Thought *for actions*, in a loop. The "Thought" lines are the model reasoning about its next move; the "Observation" lines are reality pushing back. That feedback is what makes an agent more than a fancy prompt.

## Planning — figure it all out now, or as you go?

Two ends of a spectrum for *how much* the agent plans before acting:

- **Plan-and-execute** — write a full plan first, then run the steps. Cheaper, more predictable, inspectable before running. Brittle: a rigid plan can't adapt when a step returns something unexpected.
- **ReAct-style (plan-as-you-go)** — decide each next step from what's been learned. Adapts to surprises, but can wander, repeat, or fail to converge, and costs more tokens.



Mature systems use a hybrid: plan up front for structure, re-plan when an observation breaks the plan.

## Reflection — let the agent check its own work

Reflection adds a self-critique step: after producing an answer, the model evaluates it — "is this fully supported by the retrieved chunks? did I miss anything?" — and revises if not. Reflexion (Shinn et al. 2023) is the canonical reference. It isn't free — every critique is another model call — so use it where mistakes are expensive and checkable. For a *citation-grounded* copilot, a single pass asking "is every sentence backed by a retrieved chunk?" is exactly the check that turns "plausible-sounding" into "actually grounded."

### MENTAL MODEL

Reasoning improves the *next step*. Reflection improves the *whole attempt*, after the fact. One looks forward; the other looks back. Strong agents do both.

The techniques stack rather than compete: **Chain-of-Thought** is the base trick; **ReAct** applies it to actions in a loop; **planning** sets how much to decide up front; **reflection** layers a backward-looking check on top.

### PROJECT TIE-IN

The agent's loop runs ReAct; a light up-front plan keeps it on rails; and a final reflection step — "is every claim cited?" — is the difference between a demo and something trustworthy.



## CONCEPT 3

# Protocols — MCP & A2A

**The one idea:** MCP standardises how an agent connects to *tools and data*; A2A standardises how an agent connects to *other agents*. Complementary plumbing — both exist to kill bespoke glue code.

## The problem: N×M glue

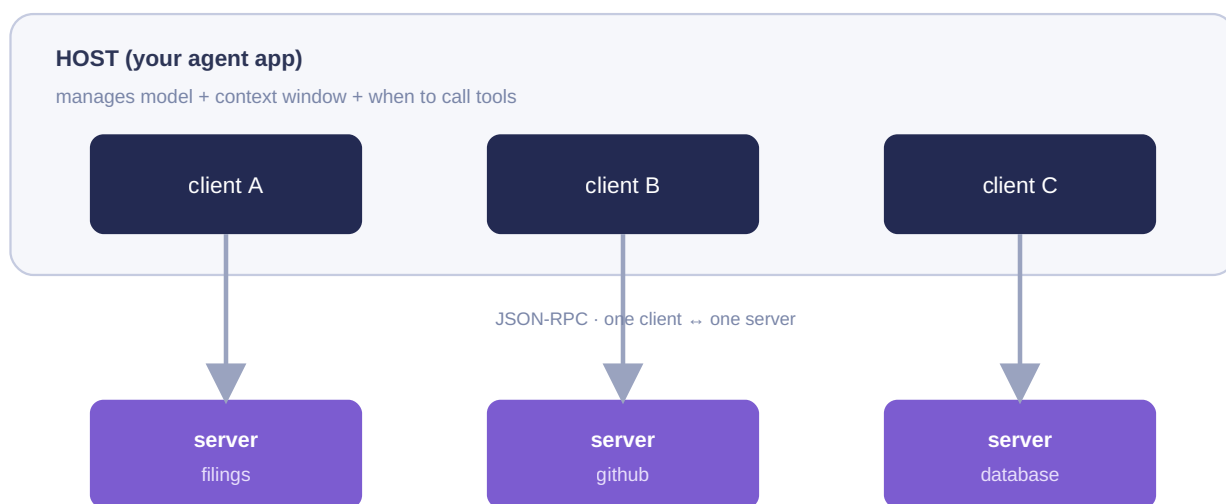
Wiring every tool to every model front-end by hand is N×M custom integrations. A protocol collapses that to N+M: build a tool once against a standard and any compliant agent can use it; build an agent once and it can talk to any compliant agent.

### MENTAL MODEL

A protocol is a standard wall socket. Without it, every appliance needs a custom wire to every power source. With it, anything with the right plug works anywhere. MCP is the socket for tools; A2A is the socket for agents.

## MCP — connecting agents to tools and data

The **Model Context Protocol**, introduced by Anthropic in late 2024 and now under Linux Foundation governance (donated to the Agentic AI Foundation in December 2025), is the open standard for connecting models to external tools, data, and prompts. Anthropic, OpenAI, Google, and Microsoft all support it. It uses a **host** → **client** → **server** architecture over JSON-RPC 2.0.



MCP host → client → server. One host runs many clients; each client holds a one-to-one connection to a server.

A server exposes three **primitives**, each with standard list/get/call methods: **Tools** (executable actions — things to *do*), **Resources** (read-only data — things to *read*), and **Prompts** (reusable templates). Every session opens with a **capability-negotiation handshake** — the client sends an `initialize` request

with its protocol version and capabilities, the server replies with its own, and version negotiation prevents mismatched peers from talking past each other.

## A2A — connecting agents to other agents

The **Agent2Agent (A2A) protocol**, announced by Google in April 2025 and donated to the Linux Foundation in June 2025, lets agents built by different teams or vendors discover each other, delegate tasks, and coordinate — without exposing their internal logic, memory, or tools. It runs over HTTP, Server-Sent Events, and JSON-RPC 2.0, and rests on a few ideas:

- ▶ **Agent Card** — a JSON metadata document describing an agent's identity, capabilities, and how to reach it. The *discovery* mechanism: a client agent reads another's card to learn what it can do.
- ▶ **Tasks** — the unit of delegated work, with a lifecycle (pending → in-progress → completed / failed). One agent hands another a task and tracks it to completion.
- ▶ **Security** — built-in auth (OAuth 2.0, API keys, mTLS) controls which agents may participate.

## MCP vs. A2A — the distinction to lock in

|                | MCP                                | A2A                                       |
|----------------|------------------------------------|-------------------------------------------|
| Connects       | agent → tools & data               | agent → other agents                      |
| Created by     | Anthropic (Nov 2024)               | Google (Apr 2025)                         |
| Governance     | Linux Foundation (AAIF)            | Linux Foundation                          |
| Transport      | JSON-RPC 2.0                       | HTTP + SSE + JSON-RPC 2.0                 |
| Discovery unit | server primitives                  | Agent Card                                |
| Work unit      | a tool call                        | a Task (with lifecycle)                   |
| Answers        | "what can this agent <i>use</i> ?" | "who can this agent <i>delegate to</i> ?" |

They're **complementary, not competing**. An orchestrator uses A2A to delegate a sub-task to a specialist agent; that specialist uses MCP to call the actual tools it needs.

### MENTAL MODEL

MCP is the agent's connection to its *hands* (tools). A2A is its connection to its *colleagues* (other agents). One agent can — and often does — use both at once.

### PROJECT TIE-IN

Module 3's project is single-agent, so MCP is the directly relevant protocol: wrap the Module 2 pipeline as an MCP server exposing `search_filings` and it becomes reusable by any host. A hand-wired tool is simpler for the build — the value of knowing MCP is that interviews ask, and it's how you'd make the tool reusable beyond this one agent. A2A only matters if you later split the copilot into cooperating agents.

## Orchestration — LangChain & LangGraph

**The one idea:** LangChain gives you the building blocks but hides control flow inside a black-box loop. LangGraph makes control flow an explicit, drawable *state machine*. For anything beyond a toy agent, you want the graph.

### LangChain — the building blocks

LangChain's value is standard abstractions you can swap without rewriting: **model wrappers** (one interface over Claude, GPT, Gemini, local), **tools** (the Module 1 function-calling shape), **prompt templates**, **chains** (fixed, deterministic sequences), and **agents** (a prebuilt ReAct loop via `AgentExecutor`).

```
from langchain.agents import create_react_agent, AgentExecutor

tools = [search_filings_tool, lookup_filing_tool, compare_numbers_tool]
agent = create_react_agent(model, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, max_iterations=6)

result = executor.invoke({"input": "Did revenue grow between the 2022 and 2023 10-K?"})
# the reason-act-observe loop runs INSIDE invoke — you don't see it
```

The catch is that last comment. The loop runs *inside* the executor. When it does something weird — repeats a tool, picks the wrong one — the decision-making is buried. Fine for a demo; the black box is the problem when you need to debug, control, and *diagram*.

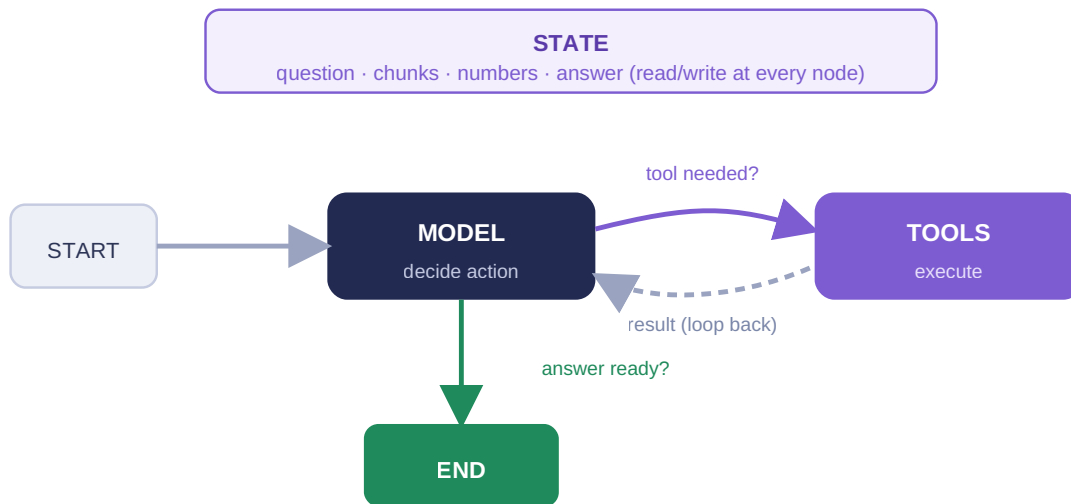
#### MENTAL MODEL

A LangChain **chain** is a fixed pipeline — the developer decides the order (like Module 2's RAG). A LangChain **agent** is a model-driven loop — the model decides the order. More flexible, but in classic LangChain, less inspectable.

### LangGraph — control flow as an explicit graph

LangGraph is the answer to the black box. Instead of hiding the loop, you *draw it*: an agent is a graph of **nodes** (steps) joined by **edges** (transitions), passing a shared **state** object around. Three concepts:

- ▶ **State** — a typed object that flows through and accumulates results (question, chunks, numbers, draft answer). Every node reads and updates it.
- ▶ **Nodes** — units of work; a function that takes state and returns updated state ("call the model," "run retrieval," "reflect").
- ▶ **Edges** — transitions. A *normal edge* always goes A → B. A *conditional edge* runs a function on the state to decide where to go — this is how you express the loop.



The conditional edge out of MODEL is the agent loop — now a labelled arrow you can point at, not a hidden `if`.

```
from langgraph.graph import StateGraph, END

graph = StateGraph(AgentState)      # AgentState = your typed state object
graph.add_node("model", call_model) # decide next action
graph.add_node("tools", run_tools)  # execute the requested tool
graph.set_entry_point("model")

def route(state):                  # conditional edge: look at state, decide
    return "tools" if state["needs_tool"] else END

graph.add_conditional_edges("model", route, {"tools": "tools", END: END})
graph.add_edge("tools", "model")    # after a tool runs, back to the model
app = graph.compile()
```

Read it against the diagram: `model` is the brain node, `tools` is the hands node, `route` is the conditional edge, and `add_edge("tools", "model")` is the loop-back. The whole control flow is four lines and it matches a picture you can draw.

#### MENTAL MODEL

LangChain's agent is a loop you *trust*. LangGraph is a loop you *draw*. Same ReAct cycle underneath — the difference is whether the control flow is hidden in a library or sitting in front of you as nodes and edges you can inspect, branch, and resume.

## What the graph buys you

Making control flow explicit fixes the production weaknesses from Concept 1: **branching** (a conditional edge routes "empty retrieval → rewrite query → retry"), **persistence & resume** (checkpoint the state object for human-in-the-loop approval), **inspectability** (look at which node ran and the state, instead of reverse-engineering a hidden loop), and a **reflection node** (the self-check from Concept 2, with a

conditional edge back to "revise" or forward to END). The tools a node calls can be hand-wired functions or MCP server tools, so the protocol and orchestration layers compose.

#### PROJECT TIE-IN

The copilot becomes a LangGraph state machine: a `model` node that plans and picks tools, a `tools` node wrapping Module 2 retrieval plus a metadata-lookup and a number-comparison tool, a conditional edge that loops until the model has what it needs, and optionally a `reflect` node that checks every claim is cited before END. Keep it small — the win condition is being able to draw the whole thing from memory, naming every node, every edge, and the condition on each edge.

---

**Papers for this module:** Chain-of-Thought (Wei et al. 2022, arXiv:2201.11903) · ReAct (Yao et al. 2022, arXiv: 2210.03629) · Reflexion (Shinn et al. 2023, arXiv:2303.11366). Read the concept files first, then the papers — the papers assume a lot, and the concepts make them click.